

Sistema de Recomendación Paralelo para E-Commerce:

Entrega 2 — Implementación Inicial y Validación Parcial

Ana María Ramírez, María José, Siloé Campos, Julio, David Mora
Computación Paralela y Distribuida
LEAD University
Prof. Johansell Villalobos Cubillo

I. INTRODUCCIÓN

Este informe documenta el avance de la Entrega 2 del sistema de recomendación paralelo para comercio electrónico, construido sobre el dataset RetailRocket (2,755,641 eventos, 1,407,580 usuarios). Se presenta evidencia funcional real de las cinco etapas del pipeline — ingesta y preprocesamiento distribuido (M1), segmentación de usuarios (M2), modelado de recomendación con feedback implícito (M3), análisis de rendimiento paralelo (M4), y un dashboard de integración final (M5) — junto con los problemas técnicos reales encontrados y resueltos en cada etapa, y las métricas de validación obtenidas sobre el dataset completo.

La Entrega 2 exige demostrar avance funcional real sobre el diseño presentado en la Entrega 1: pipeline de datos operativo, EDA completado sobre datos reales, modelo baseline entrenado con primeras métricas, primeros benchmarks de speedup, y un dashboard conectado al EDA. Este documento organiza la evidencia de cada uno de los cinco módulos del proyecto contra los cinco criterios oficiales de la rúbrica: avance funcional, organización del código, validación inicial, gestión de problemas, y documentación y presentación.

II. MÓDULO 1: INGENIERÍA DE DATOS Y PIPELINE DE INGESTIÓN

A. Avance funcional

Se implementó un pipeline ETL completo en Polars (paralelismo multinúcleo) con soporte de Dask para particionado distribuido, ejecutable de principio a fin con un solo comando. El pipeline procesa las tres fuentes del dataset RetailRocket: `events.csv`, `category_tree.csv`, y `item_properties_part1/2`. Sobre el dataset completo, el pipeline procesó **2,756,101** eventos crudos, produciendo **2,755,641** filas limpias tras la deduplicación (99.98% de retención).

B. Organización del código

El módulo `src/pipeline_datos.py` separa claramente las etapas de ingesta (`ingestar_eventos`, `ingestar_category_tree`,

`ingestar_item_properties`), `limpieza` (`limpiar_eventos`, `limpiar_category_tree`), transformación (`transformar_eventos`) y exportación a Parquet particionado, cada una documentada con docstrings en español.

C. Validación inicial

Se comparó Pandas, Polars y Dask cargando y limpiando `events.csv` completo (Tabla I).

TABLE I
BENCHMARK ETL — 2,755,641 FILAS

Herramienta	Tiempo (s)	Memoria (MB)	Speedup
Pandas	4.90	129.5	1.00×
Polars	1.62	454.3	3.02×
Dask (4 particiones)	4.01	102.3	1.22×

Polars resultó consistentemente la opción más rápida, a costa de mayor uso de memoria. Es un hallazgo relevante que, con una muestra pequeña (~900K filas), Dask resultaba *más lento* que Pandas (0.42×); al escalar al dataset completo, Dask superó a Pandas (1.22×) — confirmando empíricamente que el overhead de coordinación de Dask solo se amortiza a partir de cierto volumen de datos.

D. Gestión de problemas

Problema 1 — Archivo `events.csv` reformateado por Excel. Se detectó que el archivo original venía con separador `;` en vez de `,`, con el campo `timestamp` truncado a notación científica (p. ej. `"1,43322E+12"`), producto de haber sido abierto y reguardado por Excel. El archivo además estaba incompleto: 1,048,575 filas en vez de las ~2.76 millones esperadas. Con el timestamp colapsado a solo 477 valores únicos, la deduplicación estándar eliminó 136,164 filas (13.0%) que en su mayoría no eran duplicados reales. **Solución:** se implementó detección automática de separador y normalización de decimal, y se volvió a descargar el archivo directo de Kaggle. Verificación posterior: `2,756,101 ->`

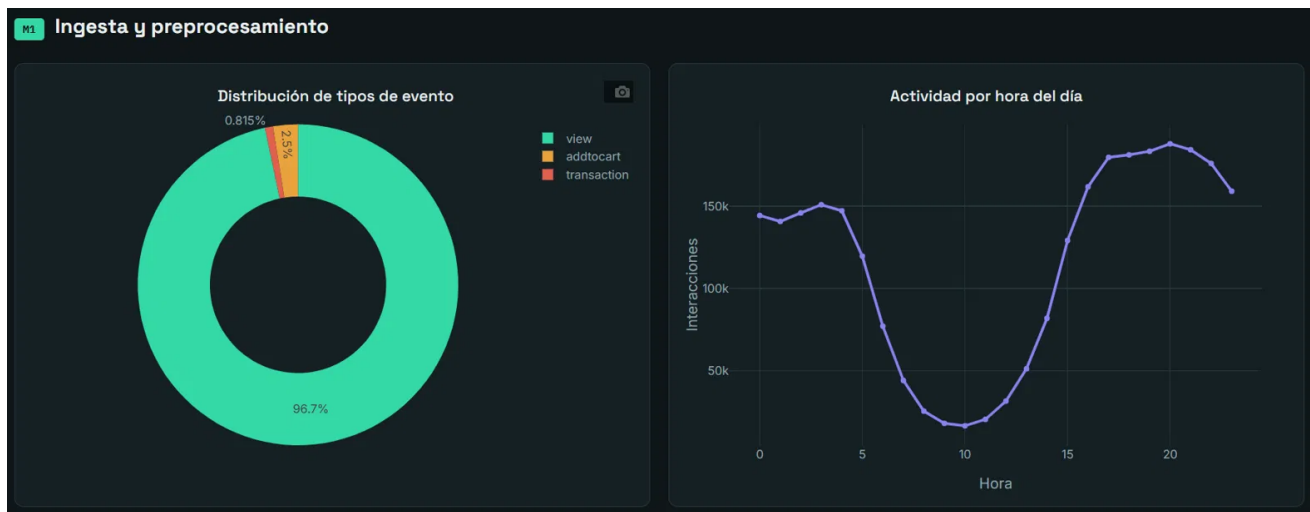


Fig. 1. Distribución de tipos de evento y actividad por hora del día, dataset completo (2,755,641 eventos).

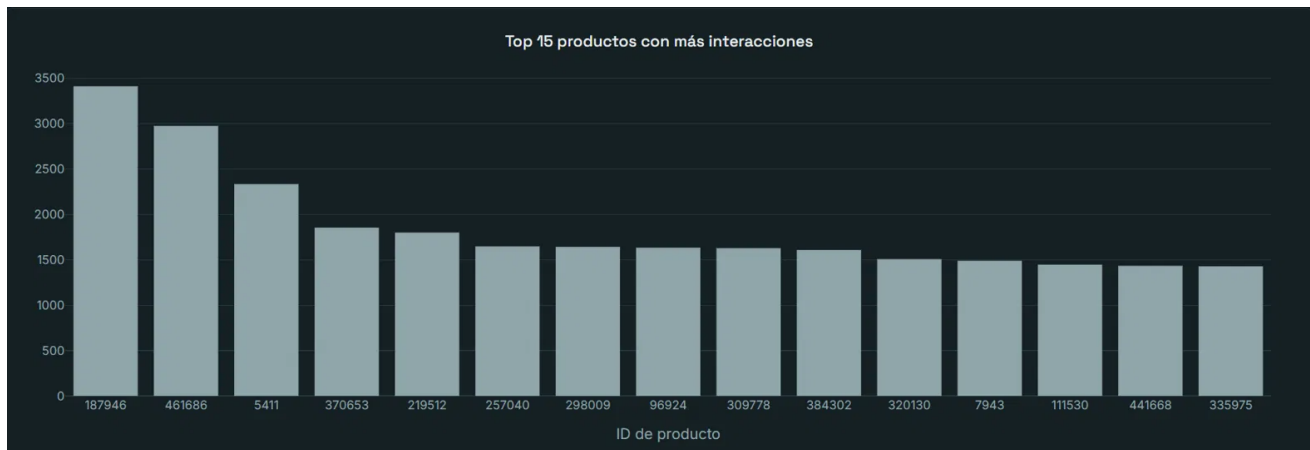


Fig. 2. Top 15 productos con más interacciones.

2,755,641 filas (460 eliminadas, 0.02%), sin alerta de calidad.

Problema 2 — item_properties_part1 en formato .xlsx. En vez de convertirlo manualmente vía Excel (repetiendo el Problema 1), se implementó lectura nativa con el motor de Polars (`pl.read_excel`), preservando precisión completa sin pasos manuales.

III. MÓDULO 2: ANÁLISIS EXPLORATORIO Y SEGMENTACIÓN DE USUARIOS

A. Avance funcional

Se realizó un EDA completo sobre los 2,755,641 eventos y 1,407,580 usuarios: estadísticas descriptivas, distribución de tipos de evento (96.7% view, 2.5% addtocart, 0.815% transaction), actividad por hora del día, y segmentación de usuarios mediante K-Means ($k = 6$, validado con método del codo y Silhouette Score) sobre variables de comportamiento agregadas por usuario (interacciones totales, productos distintos, peso implícito, conteo por tipo de evento).

B. Validación inicial

La segmentación produjo 6 clusters interpretables, con tamaños de 1,252 a 998,984 usuarios (Tabla II), exportados a `user_segments.parquet` para uso directo por M3 y M5.

TABLE II
DISTRIBUCIÓN DE USUARIOS POR CLUSTER

Cluster	Usuarios
0 — Bajo/moderado	312,873
1 — Ocasional (1 interacción)	998,984
2 — Muy activo	1,252
3 — Sobre el promedio	58,507
4 — Orientado a compra	10,501
5 — Actividad intermedia	25,463

C. Nota de alcance

La refactorización a módulo reutilizable (`src/analisis_eda.py`, con fun-

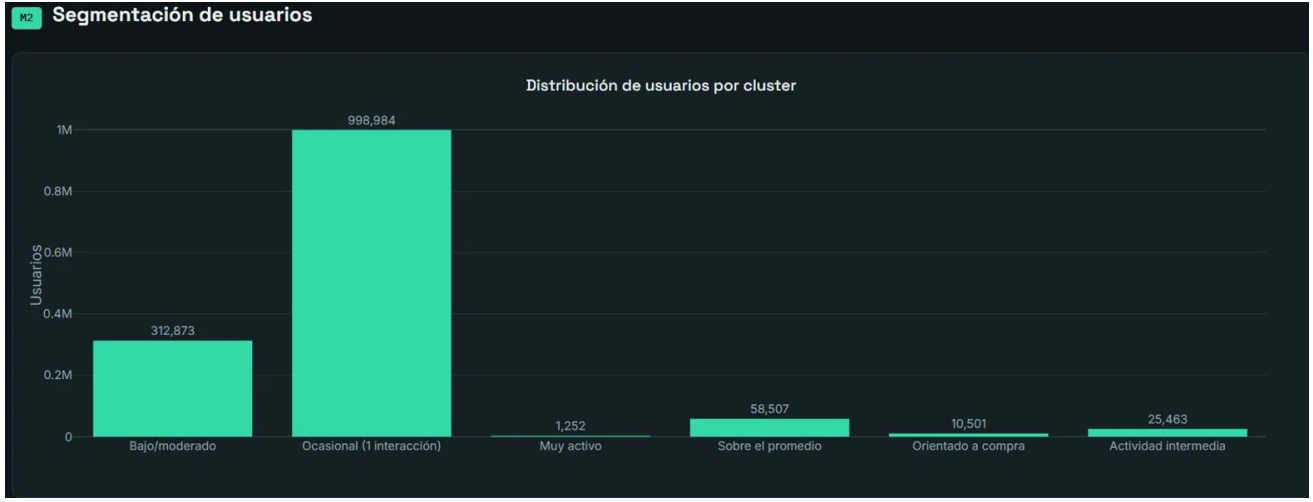


Fig. 3. Distribución de usuarios por cluster (K-Means, $k = 6$).



Fig. 4. Perfil relativo de los clusters 0, 1, 3, 4 y 5 (normalizado), con valores absolutos. El Cluster 2 (outlier) se excluye de la normalización por dominar las cinco variables simultáneamente.

ciones construir_features_usuario, segmentar_usuarios, cargar_segmentos) y la consolidación de una sección dedicada de gestión de problemas quedan como puntos de mejora para la Entrega 3.

IV. MÓDULO 3: MODELO DE MACHINE LEARNING Y RECOMENDACIÓN

A. Arquitectura y función de pérdida

El modelo baseline implementa filtrado colaborativo con feedback implícito mediante factorización matricial (ALS), vía la librería *Implicit*, siguiendo la formulación de Hu et al. [1]. Cada interacción se pondera por una señal de confianza definida en M1:

$$w(\text{evento}) = \begin{cases} 1 & \text{view} \\ 3 & \text{addtocart} \\ 5 & \text{transaction} \end{cases} \quad (1)$$

El modelo minimiza una función de mínimos cuadrados regularizada, alternando entre resolver los vectores latentes de usuario y de producto.

B. Configuración de entrenamiento

TABLE III
HIPERPARÁMETROS DEL MODELO BASELINE

Hiperparámetro	Valor
factors	64
regularization	0.01
iterations	15
División train/test	Leave-one-out temporal

La matriz de interacciones usuario-producto resultante es de $1,407,580 \times 227,081$, con 1,930,255 interacciones no nulas (densidad de 0.0006%).

C. Distribución de tiempo por etapa interna

Se midió el tiempo de cada etapa interna del pipeline de M3 (Tabla IV), lo que justifica por qué el entrenamiento y la evaluación fueron el foco del análisis de rendimiento de M4.

TABLE IV
TIEMPO POR ETAPA INTERNA DE M3

Etapa	Tiempo
Carga de datos (M1+M2, Parquet)	~2 s
División train/test	~1.6 s
Construcción de matriz dispersa	~1.5 s
Entrenamiento ALS (15 iter., 64 factores)	102.9 s
Evaluación batch (397,571 usuarios)	~280 s

El entrenamiento y la evaluación concentran, juntos, más del 95% del tiempo total de ejecución del pipeline de M3 — de ahí que el análisis de rendimiento de M4 (Sección IV) concentre el profiling en estas dos etapas.

D. Validación inicial

Evaluado sobre 397,571 usuarios (97.9% del test evaluable): Hit Rate@10 = **0.0266**, MAP@10 = **0.0124**, NDCG@10 = **0.0157** [4]. Descomponiendo por cluster de M2, el segmento más activo (Cluster 2) obtuvo el *peor* Hit Rate (0.0057) frente al Cluster 0 (0.0289) — una mayor cantidad de interacciones no implica una recomendación más fácil, dado que estos usuarios exploran un catálogo mucho más amplio.

TABLE V
HIT RATE@10 POR SEGMENTO DE USUARIO (M2)

Cluster	Hit Rate@10	Usuarios evaluados
0 — Bajo/moderado	0.0289	305,780
3 — Sobre el promedio	0.0231	57,369
5 — Actividad intermedia	0.0132	22,801
4 — Orientado a compra	0.0066	10,383
2 — Muy activo	0.0057	1,238
1 — Ocasional (1 interacción)	No evaluable (ver Sección III-D)	

E. Gestión de problemas

Cold start. El Cluster 1 (~71% de usuarios, una sola interacción) no es evaluable bajo la estrategia leave-one-out, limitación conocida del filtrado colaborativo puro.

Magnitud de los scores. Los scores del modelo son del orden de 10^{-9} – 10^{-5} según el usuario evaluado, consecuencia de la dispersión extrema de la matriz combinada con la regularización L2. Esto no afecta la validez del modelo: lo relevante es el orden relativo entre recomendaciones, no la magnitud absoluta del score.

V. MÓDULO 4: COMPUTACIÓN PARALELA Y ANÁLISIS DE RENDIMIENTO

A. Configuración experimental

Todas las mediciones siguen un protocolo de calentamiento (warm-up, 1 repetición descartada) más repeticiones promediadas, reportando media, desviación estándar, mínimo y máximo por configuración.

B. Resultados por etapa

TABLE VI
SPEEDUP Y FRACCIÓN SECUENCIAL (AMDAHL) POR ETAPA

Etapa	Speedup máx.	S (Amdahl)	Techo teórico
ETL (M1)	1.90×	54.2%	1.85×
Clustering (M2)	1.51×	60.4%	1.65×
Entrenamiento (M3)	1.09×	89.2%	1.12×

La fracción secuencial S se estimó ajustando la Ley de Amdahl [3] por mínimos cuadrados:

$$\text{Speedup}(N) = \frac{1}{S + \frac{1-S}{N}} \quad (2)$$

sobre las cuatro configuraciones medidas (1, 2, 4, 8 unidades de paralelismo).

C. Hallazgo adicional: la granularidad de partición importa más que la cantidad

Se probó la hipótesis de que la baja eficiencia del ETL se debía a un número insuficiente de particiones (5, con blocksize=16MB). Al reducir el blocksize a 4MB se generaron 23 particiones (Tabla VII).

TABLE VII
EFECTO DEL TAMAÑO DE PARTICIÓN EN EL SPEEDUP DEL ETL

Config.	Tiempo (1 worker)	Mejor caso	A partir de 4 workers
16MB (5 part.)	5.37 s	1.90× @ 8w	Sigue mejorando
4MB (23 part.)	3.81 s	1.55× @ 2w	Empeora

Con más particiones, el tiempo secuencial de base mejoró, y el óptimo se alcanzó antes (2 workers en vez de 8), pero agregar más workers después de ese punto *degradó* el rendimiento en vez de mejorarlo. Esto refuta la hipótesis simple “más particiones es mejor”: el cuello de botella real es el equilibrio entre tamaño de partición y número de workers, no la cantidad de particiones por sí sola.

D. Cuellos de botella identificados

- 1) El entrenamiento del modelo domina el tiempo total del pipeline (>90%) y es la etapa con menor margen de mejora vía paralelismo CPU ($S = 89.2\%$).
- 2) La granularidad de partición del ETL no está calibrada: al pasar de 5 a 23 particiones (blocksize de 16MB a 4MB), el punto óptimo de paralelismo se desplazó de 8 workers a 2, y agregar más workers después de ese punto *degradó* el rendimiento.
- 3) Rendimientos marcadamente decrecientes en las tres etapas: la eficiencia cae por debajo de 0.25 en las tres al llegar a 8 unidades de paralelismo.

E. Gestión de problemas

Problema 1 — Variabilidad entre repeticiones. Se observó dispersión no despreciable entre repeticiones de una misma configuración (p. ej., el ETL a 8 workers/5 particiones



Fig. 5. Ejemplo de top-10 recomendaciones generadas por el modelo ALS para un usuario de la muestra. El score es un producto punto entre vectores latentes, no una probabilidad; lo relevante es el orden relativo.



Fig. 6. Speedup máximo alcanzado por etapa del pipeline, y curva de speedup del ETL (M1) vs. número de workers, comparada contra el speedup ideal.

varió entre 2.44s y 3.36s). Esto se refleja en que las estimaciones de la fracción secuencial de Amdahl calculadas con un solo punto difieren en más de 25 puntos porcentuales según qué punto se use. Se mitiga reportando el ajuste conjunto sobre las cuatro configuraciones en vez de un único punto, y se recomienda aumentar de 2–3 a 6–8 repeticiones para la Entrega 3.

Problema 2 — cProfile no es adecuado para perfilar este entrenamiento. Se intentó perfilar `modelo.entrenar()` con `cProfile`, pero los resultados no fueron interpretables: las funciones reales de ALS mostraron tiempos acumulados cercanos a cero, mientras funciones de espera de hilos (`select`, `wait`, `acquire`) dominaron el reporte. La causa es que `cProfile` solo instrumenta el hilo principal de Python; como `Implicit` libera el GIL y ejecuta el cómputo real en hilos nativos de BLAS, esta limitación metodológica se documenta en vez de reportar resultados de profiling engañosos.

Problema 3 — Entorno de medición no aislado. Las mediciones se corrieron en una máquina de uso general, no en un clúster dedicado (p. ej. Kabré, CeNAT) ni en un contenedor con recursos reservados. Se documenta como limitación conocida; de tener acceso a un entorno dedicado, se recomienda repetir estas mediciones ahí para la Entrega 3.

VI. MÓDULO 5: DASHBOARD Y VISUALIZACIÓN

A. Avance funcional

Se implementó un dashboard interactivo (Dash/Plotly) con cuatro vistas conectadas directamente a los resultados de M1–M4: exploración de usuarios (M2), interacciones y productos (M1), recomendaciones top-N (M3), y rendimiento del pipeline (M4). Adicionalmente se generó una versión HTML estática autocontenida, sin dependencia de servidor, de la cual se extraen las Figuras 1–6.

B. Organización del código

Se estableció la estructura de carpetas `dashboard/`, `src/`, `notebooks/`, `datos/`, `resultados/` en la raíz del proyecto, con un `Makefile` que define targets reproducibles (`m1`, `m2`, `m3`, `m4`, `dashboard`, `all`).

C. Validación inicial

Se verificaron las cuatro vistas del dashboard contra datos reales, confirmando consistencia exacta con los resultados reportados en M1–M4 (Figuras 1–6).

D. Gestión de problemas

Formato de scores ilegible. Los scores de M3 se mostraban como 0.0000 con formato de 4 decimales fijos; se corrigió a notación científica explícita (ver Figura 5).

Cluster outlier en el radar chart. El Cluster 2 domina las 5 variables de perfil simultáneamente, por lo que una normalización min-max estándar no resuelve la ilegibilidad visual del resto de clusters; se separó en una visualización aparte (Figura 4).

Incompatibilidad de rutas al reorganizar carpetas. Migrar a la estructura final (`src/`, `datos/`) rompió las importaciones (`sys.path`) y las rutas de lectura de datos en los notebooks de M3, M4 y el dashboard, que aún referenciaban los nombres de carpeta anteriores (`processed/`, ruta plana sin `src/`). Se estandarizó una convención única de rutas (`RAIZ_PROYECTO`, `RAIZ_PROYECTO/"src"`, `RAIZ_PROYECTO/"datos"`) en los cinco módulos.

VII. INSTRUCCIONES DE EJECUCIÓN

- 1) `pip install -r requirements.txt`
- 2) Colocar los archivos originales de RetailRocket en `Datasets/`.
- 3) Ejecutar en orden: `notebooks/PipelineM1.ipynb`
→ `M2_Analisis_EDA.ipynb` →
`M3_Modelo_Recomendacion.ipynb` →
`M4_Analisis_Rendimiento.ipynb`.
- 4) Dashboard: `cd dashboard && python app.py` (interactivo) o `python generar_dashboard_html.py` (HTML estático).
- 5) Alternativamente, `make all` ejecuta el pipeline completo de punta a punta.

VIII. CONCLUSIONES PARCIALES

La Entrega 2 demuestra un pipeline funcional de principio a fin sobre el dataset completo de RetailRocket, con evidencia cuantitativa en cada etapa. El hallazgo más relevante del análisis de rendimiento — que la etapa dominante en tiempo (entrenamiento) es también la que menos se beneficia del paralelismo CPU ($S = 89.2\%$, techo de $1.12\times$) — justifica empíricamente la decisión, ya planteada en la justificación técnica de la Entrega 1, de explorar aceleración GPU (RAPIDS cuML / PyTorch) en la Entrega 3, en lugar de continuar optimizando exclusivamente por la vía de más paralelismo CPU.

REFERENCES

- [1] Y. Hu, Y. Koren, and C. Volinsky, “Collaborative filtering for implicit feedback datasets,” in *Proc. 2008 Eighth IEEE Int. Conf. Data Mining (ICDM)*, 2008, pp. 263–272.
- [2] Y. Zhou, D. Wilkinson, R. Schreiber, and R. Pan, “Large-scale parallel collaborative filtering for the Netflix Prize,” in *AAIM 2008*, LNCS vol. 5034, 2008, pp. 337–348.
- [3] G. M. Amdahl, “Validity of the single processor approach to achieving large scale computing capabilities,” in *Proc. AFIPS Spring Joint Computer Conf.*, 1967, pp. 483–485.
- [4] K. Järvelin and J. Kekäläinen, “Cumulated gain-based evaluation of IR techniques,” *ACM Trans. Inf. Syst.*, vol. 20, no. 4, pp. 422–446, 2002.
- [5] M. Rocklin, “Dask: Parallel computation with blocked algorithms and task scheduling,” in *Proc. 14th Python in Science Conf.*, 2015, pp. 126–132.
- [6] Retailrocket, “Retailrocket recommender system dataset,” Kaggle, 2022. [Online]. Available: <https://www.kaggle.com/datasets/retailrocket/ecommerce-dataset>